LAB 7

Fuzz Testing

Coupon Validator

Name	Ashritha Kamalakar Hegde
SRN	PES1UG24CS089
Programme / Branch	B.Tech – Computer Science and Engineering
Course / Lab	Software Engineering Laboratory

1. Objective

To use Hypothesis-based fuzz testing to generate inputs for the coupon validator functions, identify failures caused by unexpected or invalid inputs, and modify the application to handle such inputs safely.

2. Software and Environment

Item	Details
Operating system	Windows
Python	3.14.0
Testing framework	pytest 9.2.0
Property-based testing library	Hypothesis 6.170.15
Application file	coupon_validator.py
Test file	test_coupon_validator.py

3. Functions Under Test

Function	Purpose	Input
split_code()	Splits a coupon code into its prefix and numeric value.	Coupon-code string
most_expensive()	Returns the highest price from comma-separated values.	CSV price string
expiry_days()	Calculates calendar days since 1 January 2024.	Date in YYYY-MM-DD format

4. Procedure and Results

4.1 Original application

The original coupon_validator.py application was executed with valid sample inputs.

- Coupon code: save-20
- Prices: 199,49.5,320
- Expiry date: 2025-01-01

```
D:\sem5\SE\L6_L7_31_coupon_validator\lab7>python coupon_validator.py
==== Running split_code ====
Enter a coupon code (e.g. save-20): save-20
Prefix and Value: ('SAVE', 20)

==== Running most_expensive ====
Enter item prices as CSV (e.g. 199,49.5,320): 199,49.5,320
Most Expensive Item: 320.0

==== Running expiry_days ====
Enter an expiry date as YYYY-MM-DD: 2025-01-01
Days Since 2024: 366

D:\sem5\SE\L6_L7_31_coupon_validator\lab7>
```

Figure 1. Output of the original coupon validator application.

4.2 Fuzz testing the original code

Hypothesis strategies st.text() and st.none() generated arbitrary strings and None values for each function. The initial test run reported failures in all three functions.

```
D:\sem5\SE\L6_L7_31_coupon_validator\lab7>python -m pytest test_coupon_validator.py
============================= test session starts =============================
platform win32 -- Python 3.14.0, pytest-9.2.0, pluggy-1.7.0
rootdir: D:\sem5\SE\L6_L7_31_coupon_validator\lab7
plugins: anyio-4.11.0, hypothesis-6.170.15
collected 3 items

test_coupon_validator.py FFF                                             [100%]

================================== FAILURES ===================================
_____________________ test_split_code_no_crash ______________________________
  + Exception Group Traceback (most recent call last):
  |   File "D:\sem5\SE\L6_L7_31_coupon_validator\lab7\test_coupon_validator.py", line 6, in test_split_code_no_crash
  |     def test_split_code_no_crash(s):
  |                    ^^^
  | ExceptionGroup: Hypothesis found 3 distinct failures. (3 sub-exceptions)
  +-+---------------- 1 ----------------
    | Traceback (most recent call last):
    |   File "D:\sem5\SE\L6_L7_31_coupon_validator\lab7\test_coupon_validator.py", line 7, in test_split_code_no_crash
    |     split_code(s)
    |     ^^^^^^^^^^^^^
    |   File "D:\sem5\SE\L6_L7_31_coupon_validator\lab7\coupon_validator.py", line 3, in split_code
    |     return prefix, int(value)
    |                    ^^^^^^^^^^
    | ValueError: invalid literal for int() with base 10: ''
    | Failing test case: test_split_code_no_crash(
    |     s='-',
    | )
    +---------------- 2 ----------------
    | Traceback (most recent call last):
    |   File "D:\sem5\SE\L6_L7_31_coupon_validator\lab7\test_coupon_validator.py", line 7, in test_split_code_no_crash
    |     split_code(s)
    |     ^^^^^^^^^^^^^
    |   File "D:\sem5\SE\L6_L7_31_coupon_validator\lab7\coupon_validator.py", line 2, in split_code
    |     prefix, value = code.strip().upper().split('-')
    |     ^^^^^^^^^^^^
    | ValueError: not enough values to unpack (expected 2, got 1)
    | Failing test case: test_split_code_no_crash(
    |     s='',  # or any other generated value
    | )
    +---------------- 3 ----------------
    | Traceback (most recent call last):
    |   File "D:\sem5\SE\L6_L7_31_coupon_validator\lab7\test_coupon_validator.py", line 7, in test_split_code_no_crash
    |     split_code(s)
    |     ^^^^^^^^^^^^^
    |   File "D:\sem5\SE\L6_L7_31_coupon_validator\lab7\coupon_validator.py", line 2, in split_code
    |     prefix, value = code.strip().upper().split('-')
    |                     ^^^^^^^^^^
    | AttributeError: 'NoneType' object has no attribute 'strip'
    | Failing test case: test_split_code_no_crash(
    |     s=None,  # or any other generated value
    | )
    +------------------------------------
```

Figure 2. Hypothesis failures for split_code() and most_expensive().

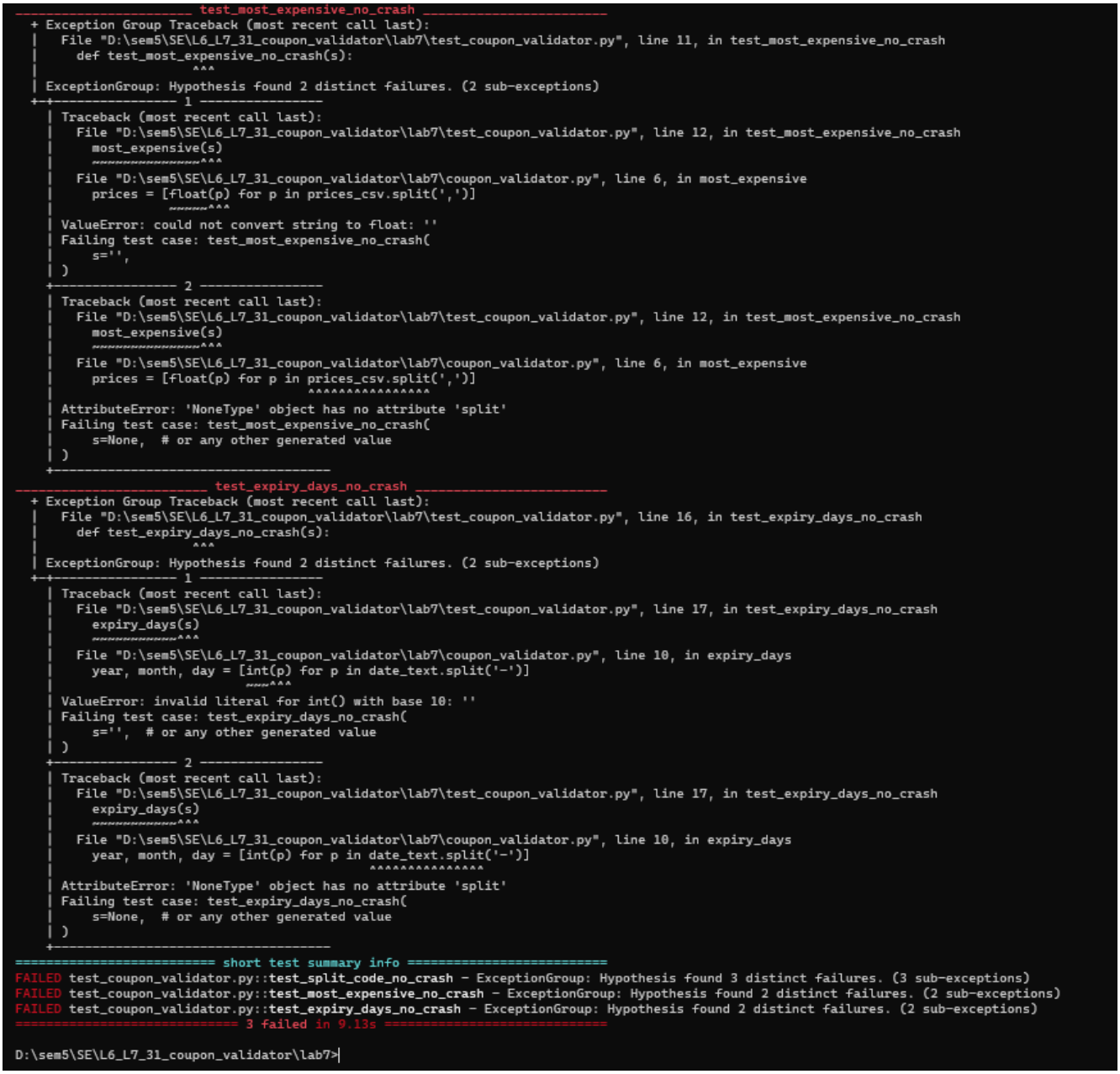


Figure 3. Hypothesis failures for expiry_days() and the test summary.

4.3 Input validation fixes

The application was updated to validate input types and formats, handle conversion errors, reject invalid values, and return None for invalid inputs instead of raising exceptions.

Function	Changes made
split_code()	Checks input type, exactly one hyphen, non-empty fields, and integer conversion.
most_expensive()	Checks input type, empty CSV fields, float conversion, and finite numeric values.
expiry_days()	Checks input type and YYYY-MM-DD calendar validity; uses actual date arithmetic.

4.4 Tests after fixing the code

The updated test file contains three Hypothesis fuzz tests and three valid-input assertions. The final pytest run completed successfully.

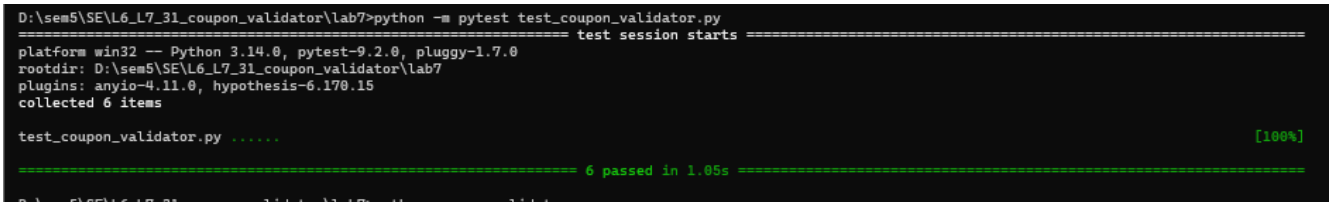


Figure 4. Final pytest result showing all six tests passing.

4.5 Corrected application

The corrected application was executed again using the same valid sample inputs.

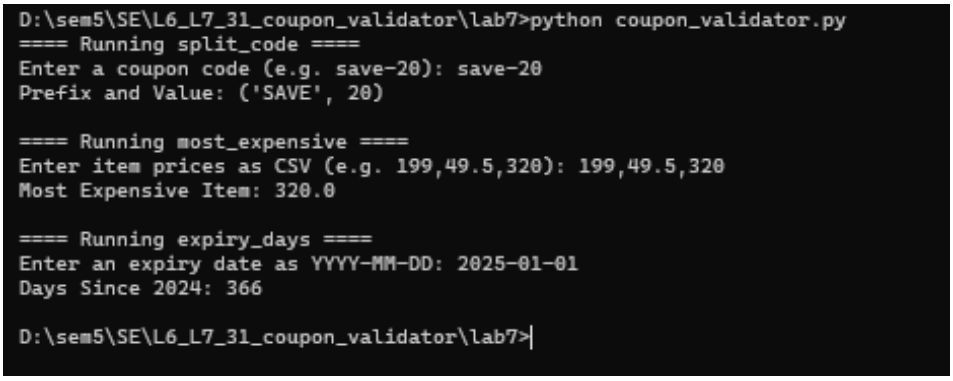


Figure 5. Output of the corrected coupon validator application.

5. Conclusion

Hypothesis-based fuzz testing exposed input-handling failures in all three coupon validator functions. After adding validation and safe error handling, the application completed the fuzz tests and valid-input checks successfully, with 6 tests passing.

6. Files Submitted

- coupon_validator.py – corrected application
- test_coupon_validator.py – Hypothesis fuzz tests and valid-input tests
- Lab 7 report PDF – screenshots and results